

UNIDAD 1

Que es la ingeniería de software?

Primero, el software no es solo código ejecutable, sino también programas, datos, reglas, procedimientos, documentación y otros artefactos necesarios para operar y mantener un sistema. Conocimiento representado en distintos niveles de abstracción: necesidades del negocio, requerimientos, modelos, diseño, código y pruebas.

Ingeniería de Software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software. Desde concebir el problema hasta operar y evolucionar el producto. No busca solo que el software funcione hoy, sino también que satisfaga necesidades reales, tenga calidad, pueda cambiar y se entregue con un uso responsable de tiempo y recursos.

El software se diferencia de la manufactura. Cada solución tiene mucho trabajo intelectual propio, copiarlo cuesta poco pero desarrollarlo cuesta mucho, no se desgasta pero se deteriora por cambios en el entorno, y sus fallas pueden provenir de conceptos equivocados además que de errores de codificación.

El termino surge en conferencias de la OTAN en 1968 1969 por la cantidad de proyectos que crecían en tamaño y complejidad más rápido que la capacidad de gestionarlos.

Crisis del software: Programar bien no alcanza, hay que entender las necesidades, coordinar personas, estimar, gestionar cambios, probar y aprender. Ayudan mucho la comunicación con el usuario, requerimientos claros, planificación realista y personas competentes, y perjudican requerimientos incompletos o cambiantes y expectativas irreales.

Ariane 5, vuelo 501: Se perdió por errores de especificación y diseño en el software de referencia inercial. Las pruebas y revisiones no cubrieron suficientemente la interacción del sistema completo.

Mars Climate Orbiter: Se perdió por incompatibilidad entre sistemas métrico e imperial.

Apollo 11: Cumplido su función durante la misión, dentro de un programa más amplio de ingeniería, integración y operaciones.

No solo el código importa, también la arquitectura, interfaces, pruebas, coordinación y objetivos.

Disciplinas de la IS

- Construcción: R A D I P D M. Cada etapa transforma conocimiento y produce artefactos verificables.
- Gestión: Planificación, estimación, seguimiento, control de cambios, riesgos, recursos y métricas. Ordena decisiones bajo restricciones de alcance, tiempo y costo.
- Soporte y calidad: SCM, aseguramiento de calidad, verificación y validación. Actividades transversales.

Proceso, proyecto, producto y ciclos de vida

Un proceso de software es el conjunto organizado de actividades, métodos, practicas, personas y herramientas para desarrollar o mantener software. Pauta reutilizable de cómo trabajar.

Un proyecto es una iniciativa temporal, única, con objetivos y duración fija.

El producto es el resultado que se usa, opera y evoluciona.

Proceso adaptado + personas y herramientas -> Proyecto -> Producto

El ciclo de vida del proyecto va desde el inicio hasta su cierre.

El ciclo de vida del producto incluye idea, desarrollo, operación, mantenimiento, nuevas versiones y retiro. Suele superar la duración del proyecto, y puede atravesar varios de estos.

Un modelo de ciclo de vida o de proceso representa las fases y el orden de trabajo. No equivale al proceso completo. El modelo organiza el flujo, el proceso especifica actividades, responsabilidades, controles, etc.

Procesos definidos y empíricos

Un proceso definido supone que entradas y actividades suficientemente estables permiten predecir resultados parecidos al repetir el procedimiento. Se especifica el trabajo por adelantado y se controla el cumplimiento. Adecuado cuando la tecnología, requerimientos y entorno son conocidos y la incertidumbre es baja.

Trazabilidad y previsibilidad. Si la incertidumbre es alta, el plan inicial puede quedar obsoleto.

Un proceso empírico reconoce que en problemas complejos los resultados no son totalmente predecibles. Se controla mediante transparencia, inspección y adaptación, los tres pilares del empirismo. Apropiado cuando hay un entorno de alta incertidumbre y muchos cambios.

Respuesta temprana al error. Exige disciplina, feedback real y capacidad de decisión. Empírico no significa improvisado.

Modelos de ciclo de vida: Ventajas, límites y elección

- Secuencial o cascada: R -> A/D -> C -> P -> Entrega, con hitos y entregables por fase. Facilita presupuesto, contratos y control cuando requisitos y tecnología son estables. Detección de error tardía, y costo de arreglarlo elevado. No es imposible volver atrás, solo es más difícil.

- Modelo en V: Variante secuencial que relaciona cada nivel de especificación con su nivel de prueba. Aceptación frente a necesidades, sistema frente a requerimientos, integración frente a arquitectura, unidad frente a componentes. Aporta trazabilidad y planificación temprana de pruebas. Conserva el problema de la validación tardía si no se usan prototipos.

- Iterativo: Se desarrolla una solución inicial y se revisa en ciclos. Permite descubrir errores, mejorar diseño y ajustar requisitos, pero requiere gestionar retrabajo y evitar iteraciones sin objetivo verificable.

- Incremental: Se entrega el producto por porciones funcionales completas. Permite valor temprano y priorización. Requiere arquitectura integrable, interfaces estables y coordinación de dependencias. Muchos proyectos son iterativos e incrementales.

- Prototipado: Un prototipo explora requisitos, interacción o viabilidad antes de comprometer toda su construcción. Reduce incertidumbre. El riesgo es tratar al prototipo como el producto final, o que una interfaz convincente se interprete como calidad interna.

- Espiral orientada a riesgos: Cada iteración fija objetivos, analiza riesgos, construye/prototipa y evalúa con los interesados. Conviene en proyectos grandes o costosos. Control explícito del riesgo.

Enfoques ágiles como scrum usan iteraciones cortas, un backlog priorizado y revisión frecuente de incrementos útiles. Responden bien a incertidumbre de necesidades y permiten reordenar trabajo. Necesitan colaboración sostenida, calidad técnica y disponibilidad de los interesados. Scrum es un marco de gestión empírica.

Criterios de elección: Estabilidad de requerimientos – Costo de cambiar – Riesgo técnico, legal y de seguridad – Necesidad de entrega temprana – Disponibilidad de usuarios – Tamaño y distribución del equipo – Incertidumbre.

Componentes y administración de un proyecto de software

Un proyecto requiere objetivos claros y alcanzables, patrocinio, stakeholders, un equipo, alcance, proceso y ciclo de vida adaptados, plan, recursos, estimaciones, controles y criterios de calidad. El líder coordina, el equipo comparte responsabilidad por los resultados. El plan responde que, como, quien, cuando.

Alcance del producto: Características y funcionalidades. Se comprueba contra los requerimientos.

Alcance del proyecto: Trabajo necesario para entregarlo. Se comprueba contra el plan.

Triple restricción: Alcance – Tiempo – Costo. Idealmente en ágil, la calidad no se negocia.

Planificación: Definir alcance, elegir proceso y ciclo de vida. Estimar tamaño, esfuerzo, calendario y costo, analizar riesgo, asignar trabajo, establecer controles y métricas.

Seguimiento: Comparar plan y realidad, analizar variaciones, decidir acciones.

Riesgo: Evento incierto con posible impacto. Se identifica, se evalúa por probabilidad e impacto, se planifica una respuesta y se revisa durante todo el proyecto.

Métricas: De producto, de proyecto, de proceso. Deben tener un motivo y medir algo relevante que aporte al éxito.

Estimación clásica de software: Pronosticar con incertidumbre. Planificar es decidir qué se hará con ese pronóstico. Estimar no es comprometerse. Distinguir tamaño del producto, esfuerzo, duración, costo y recursos. Más personas no significa avanzar más rápido.

Estimación ágil?

No Silver Bullet

Ninguna tecnología o técnica de gestión, por si sola, prometía una mejora de un orden de magnitud en productividad, fiabilidad y simplicidad del software dentro de una década.

No es que no haya progreso, sino que este se viene logrando por mejoras continuas.

Se distingue la esencia del accidente. Esencia es el trabajo de concebir, especificar, diseñar y validar estructuras conceptuales complejas. Lo accidental es la dificultad de representar esas ideas en lenguajes y máquinas concretas, sintaxis compleja, memoria escasa. Si el trabajo accidental no representa un 90% del esfuerzo, no puede mejorar en x10 atacando esos problemas

Cuatro dificultades esenciales: Complejidad – Conformidad – Cambiabilidad – Invisibilidad
Lenguajes de alto nivel facilitaron programar, se redujo la espera de ejecución, IDEs facilitaron herramientas.

Ninguna de las silver bullets candidatas fueron suficientes. Todas pueden ayudar pero no eliminan la necesidad de decidir correctamente qué construir, ni la complejidad inherente del diseño

4 ideas prometedoras que atacan la esencia: Reutilizar código cuando una solución existente satisface la necesidad – Refinar requisitos mediante prototipos rápidos – Hacer crecer el software incrementalmente – Identificar y desarrollar grandes diseñadores.

UNIDAD 2

Gestión de producto: construir valor

Decide para quién, qué problema y por qué se construye. Formula hipótesis sobre usuarios, prioriza oportunidades, define hoja de ruta y mide resultados.

Output es lo entregado.

Outcome es el resultado, cambio observable que produce, como más ventas o menor tiempo en x pantalla.

Construir -> Medir -> Aprender (de lean) busca reducir el riesgo de desarrollar algo que nadie necesita.

Hipótesis, experimento más pequeño que pueda refutarla, se observa conducta real, se preserva ajusta o pivotea.

Hipótesis de valor: El producto resuelve un problema?

Hipótesis de crecimiento: Como los usuarios descubren y adoptan el producto?

MVP: Versión mínima del producto que permite aprender algo relevante sobre la propuesta de valor con usuarios reales. Sin funciones superfluas, no incompleto ni inseguro. Debe producir evidencia sobre una hipótesis de negocio. Video de Dropbox.

MMF: Funcionalidad mínima comercializable. Aporta valor en sí misma.

MVF: Funcionalidad mínima viable. Explora un área incierta dentro de un producto existente.

MMP: Producto mínimo comercializable. Reúne lo suficiente para un primer lanzamiento.

MMR: Lanzamiento mínimo comerciable.

MLP: Producto mínimo amable. Busca experiencia memorable, menor fricción y conexión emocional.

Filosofía ágil y requerimientos ágiles

Personas e interacciones – Procesos y herramientas

Software funcionando – Documentación exhaustiva

Colaboración con el cliente – Negociación contractual

Respuesta al cambio – Seguir un plan rígido

Entregas frecuentes, cooperación diaria entre negocio y desarrollo, excelencia técnica, simplicidad, equipos autoorganizados, reflexión periódica. Sostener ciclos de aprendizaje y calidad.

Un requerimiento de negocio expresa un objetivo (reducir un 20% el tiempo de atención).

Uno de usuario, una capacidad deseada (consultar estado de un trámite).

Uno funcional, comportamiento del sistema (generar comprobante).

Uno no funcional, una cualidad o restricción medible.

Uno de implementación, una decisión técnica impuesta.

Historias de usuario

Recordatorio de conversación sobre una necesidad valiosa. No es una especificación técnica ni una tarea. Card (descripción breve) – Conversation (detalles y alternativas) – Confirmation (criterios y pruebas).

Como [rol] quiero [capacidad] para [beneficio]. El beneficio obliga a discutir valor.

La US se evalúa con INVEST. Independent Negotiable Valuable Estimable Small Testable

Conviene US en porciones verticales que atraviesen interfaz, lógica, datos, y produzcan una capacidad observable. Separarlas por capas produce tareas técnicas, pero no US completas.

Modelado de roles: Identificar grupos con objetivos y comportamientos distintos. Usar tarjetas de rol o personas. Un proxy puede dar info cuando el usuario no está disponible, pero introduce riesgo de suponer necesidades equivocadas. Las US deben conservar la perspectiva del usuario.

Criterios de aceptación: Fijar límites específicos de una historia y ayudan a que el PO, desarrolladores y testers compartan entendimiento.

Prueba de aceptación: Ejecuta casos para comprobar los criterios.

Definition of ready: Condiciones o acuerdo opcional del equipo sobre la claridad suficiente para iniciar el trabajo.

Definition of done: Estándar de calidad compartido que debe cumplir un incremento para considerarse terminado. Una US puede cumplir sus criterios particulares y no considerarse done si no paso pruebas, integración o documentación requeridas para la DoD.

Spike: Investigación acotada en tiempo para reducir una incertidumbre importante antes de estimar o implementar. Puede ser técnica (soporta la API el volumen previsto?) o funcional (los usuarios entienden la visualización?). Debe tener una pregunta, un límite y un resultado verificable.

Estimaciones ágiles

Son estimaciones ágiles porque usan términos relativos en lugar de absolutos.

Un SP es la unidad propia del equipo para valorar tamaño considerando complejidad, esfuerzo e incertidumbre. 8 SP no significa nada en sí mismo.

Mantener una US canónica de referencia, y una escala estable (Fibonacci 1 2 3 5 8 13).

Si una US es demasiado grande o tiene mucha complejidad, dividirla.

Poker planning: El equipo escucha la US, preguntan al PO, estiman en privado y comparan simultáneamente. Los extremos explican su razonamiento y el equipo se pone de acuerdo en un valor para la US. El objetivo es lograr el entendimiento de la US.

Velocidad: Suma de SP de US terminadas en un sprint. Las parciales cuentan 0. Sirve para pronosticar dentro del equipo.

Capacidad: Tiempo de trabajo disponible después de licencias, reuniones y obligaciones.

Scrum

Marco liviano para generar valor en problemas complejos mediante empirismo. Transparencia, inspección y adaptación.

Cinco valores: Compromiso, foco, apertura, respecto y coraje.

Equipo pequeño, multifuncional y autogestionado.

Responsabilidades del scrum team

Product Owner: Maximiza el valor y es responsable del Product backlog y su orden. Puede delegar tareas pero conserva la responsabilidad.

Scrum master: Ayuda a eliminar impedimentos y mejorar el marco de trabajo.

Developers: Crean cada sprint un incremento utilizable que cumple la DoD, planifican el trabajo y adaptan el sprint backlog.

Stakeholders colaboran y dan feedback.

Sprint: Contenedor de todos los eventos, de duración máxima de 4 semanas, con un Sprint goal que da dirección. Dentro del sprint pueden aclararse y renegociarse detalles del alcance con el PO, siempre preservando el objetivo y calidad. Cada sprint debe producir al menos un incremento utilizable.

Eventos y tiempos máximos de la guía 2020

Sprint planning: 2 horas por semana planificada. Responde por qué vale la pena el sprint, qué puede hacerse y cómo se hará.

Daily scrum: 15 minutos. Permite a devs inspeccionar avance hacia el sprint goal y adaptar el plan.

Sprint review: Hasta 4 horas. Inspecciona resultado y contexto con stakeholders y adapta el producto backlog.

Sprint retrospective: Hasta 3 horas. Mejora calidad y efectividad del equipo.

El refinamiento del producto backlog es una actividad continua, no un evento formal.

Artefactos y compromisos

Product backlog: Lista emergente, ordenada y transparente. <--> Product goal: Objetivo a largo plazo.

Sprint backlog: Sprint goal, items elegidos y plan. <--> Sprint goal.

Increment: Trabajo integrado y utilizable <--> DoD: Estándar de calidad.

Si un ítem no cumple el DoD, no integra al incremento y vuelve al backlog.

Planificación en varios niveles: Estrategia -> Producto -> Release -> Sprint -> Día

Se definen objetivos a nivel de producto, se proyectan capacidades y riesgo a nivel de release, en la

sprint planning se selecciona trabajo para un futuro cercano, y en la daily se adapta el plan del día. Cuanto más se acerca el trabajo, más granular se vuelve.

Métricas útiles: Incremento funcionando y probado, resultados del uso, capacidad, velocidad, burndown (trabajo restante). Running Tested Features expresa mejor el progreso que tareas casi cumplidas.

Ampliación 2026

Product Developers. Crean el incremento.

Stakeholders y Supporters.

Definition of Output Done y Definition of Outcome Done.

Admite ayuda de IA bajo responsabilidad humana.

UNIDAD 3

Propósito y conceptos básicos de SCM

Disciplina transversal que identifica los elementos del software, controla sus cambios, registra su estado y verifica la integridad de las configuraciones durante todo su ciclo de vida.

Saber qué versión de qué artefacto conforma un producto, por qué cambió, quién autorizó el cambio y cómo reproducir una entrega. Abarca decisiones, trazabilidad y auditorías.

Un IC es cualquier artefacto del producto o proyecto que deba compartirse, cambiarse y seguirse (requerimientos, arquitectura, código, casos de prueba, scripts, dependencias, config de despliegue, manuales, planes o registro de defectos).

Versión: Estado particular de un IC en un momento.

Variante: Línea alternativa que evoluciona por separado. Ej.: una edición para otro sistema operativo.

Configuración del software: Conjunto específico de IC y versiones que forman un sistema en un momento dado.

Repositorio: Conserva IC, historia, atributos y relaciones, y permite evaluar el impacto de cambios. En un repo centralizado hay una copia principal en un servidor, el control es directo pero el server es un punto crítico. En uno distribuido, cada clon contiene la historia completa, favoreciendo el trabajo local.

Línea base: Configuración revisada y acordada que sirve como referencia. Cambiarla requiere un procedimiento controlado. No es simplemente la última versión, ni un release comercial. Una etiqueta puede identificarla para reconstruir el estado.

Release: Paquete entregado a usuarios o a producción, que debe poder vincularse con una línea base, artefactos, pruebas y notas de versión.

Branch: Permite desarrollo paralelo o experimental

Merge: Integra cambios. Si dos ramas modificaron el mismo contenido incompatible, aparece un conflicto. Ramas muy largas aumentan divergencia y costo de integración.

Actividades fundamentales de SCM

- Identificación de configuración: Definir que IC se controlan, sus nombres unívocos, estructura,

relaciones y criterios de versionado.

- Control de cambios: Una solicitud describe motivo y artefactos afectados. Se analiza impacto en requisitos, interfaces, costos, plazo, pruebas y riesgos. Se aprueba, rechaza o pospone. Se implementa y se revisa. Se actualiza la baseline y se informa la decisión.
- Comité de control de cambios: Para entornos formales. Representantes de áreas involucradas. Un hotfix no requiere reunión, pero un cambio a la seguridad sí.
- Registro e informe de estado: Responder qué IC existe, qué versión está aprobada, que cambios están pendientes, que release los contiene y que pruebas los validaron. La trazabilidad conecta solicitud -> registro -> cambio de código -> prueba -> entrega.
- Auditoria de configuración: La funcional (FCA) verifica que comportamiento y rendimiento reales se correspondan con los requisitos. La física (PCA) comprueba que artefactos, versiones y documentación declarados estén realmente presentes y consistentes. Ambas complementan verificación y validación.
- Plan de SCM: Establece el alcance de IC, convenciones, herramientas, responsabilidades, flujo de cambios, regla de ramas y baselines, auditorias, informes y procedimientos de entrega. Sin esto aparecen pérdida de cambios, superposición de trabajo y versiones imposibles de reconstruir.

SCM en ambientes ágiles

La agilidad necesita SCM ligera, visible y automatizada. Integración frecuente, pruebas automáticas, revisión de cambios, etc. La gestión de configuración se integra al trabajo del sprint y la DoD. Reaccionar rápido no es aceptar todo sin conocer su impacto. Un CCB formal puede ser sustituido o simplificado para cambios ordinarios.